IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 13 January 2026, accepted 11 February 2026, date of publication 9 March 2026, date of current version 25 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3672373

APPLIED RESEARCH

UMLRev: A Context-Aware Feature-Oriented Approach to Support Continuous Reverse Engineering of UML Models

KLEINNER FARIAS, LUCAS SCHENATTO, CRISTIANO FARIAS, WILLIAN BOLZAN, ROBSON KEEMPS, AND CARLOS CARBONERA

Universidade do Vale do Rio dos Sinos (UNISINOS), São Leopoldo, Rio Grande do Sul 93022-750, Brazil

Corresponding author: Kleinner Farias (kleinnerfarias@unisinos.br)

This work was supported in part by the Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) under Grant 312320/2025-6.

ABSTRACT Maintaining up-to-date design models is a persistent challenge in modern software development, especially within continuous integration (CI) environments. While prior research in UML reverse engineering has offered valuable insights, existing approaches often neglect contextual information about application features and lack integration with automated delivery pipelines. To address this gap, we present UMLRev, a context-aware feature-oriented approach to support continuous reverse engineering of UML models. UMLRev proposes a practical tool that supports the continuous reverse engineering of UML models from feature-level execution traces. UMLRev integrates directly into CI pipelines and automatically generates UML class and sequence diagrams for every software release, ensuring that documentation evolves alongside the codebase with minimal developer effort. Unlike previous approaches, UMLRev combines context-awareness, feature orientation, and CI compatibility into a unified, working prototype. Features are manually delimited via test scripts, allowing teams to define and track behaviorally meaningful units over time. We conducted a qualitative study with 22 industry professionals to evaluate the tool's perceived usefulness, ease of use, and impact on UML adoption. Results show that 91% found UMLRev helpful in generating diagrams in agile settings, and over 70% believe it encourages broader UML use. These findings underscore the tool's potential to close the gap between design documentation and evolving implementations in agile software practices.

INDEX TERMS Reverse engineering, UML models, continuous integration.

I. INTRODUCTION

Maintaining accurate and up-to-date software design models remains a significant challenge in modern development, especially in fast-paced environments driven by continuous integration (CI) pipelines [1], [2]. While software modeling offers essential cognitive support for comprehension, maintenance, and evolution [3], [4], practitioners often face difficulties keeping UML diagrams current [5], [6]. Prior studies [7], [8] confirm that missing, outdated, or informal documentation increases the risk of design flaws, potentially leading to costly rework or architectural erosion. Although reverse engineering techniques have sought to bridge this gap [9], existing tools struggle to generate context-rich diagrams that align with evolving implementations and often fail to scale across iterative delivery cycles.

UML behavioral and structural diagrams remain vital artifacts in industrial settings. They facilitate program comprehension by visually conveying the system's architecture and runtime behavior [3], reduce cognitive effort for newcomers and maintainers, and support impact analysis during system evolution [4]. Empirical evidence suggests that the presence of these diagrams significantly improves functional correctness and design quality during maintenance tasks. However, their practical use has declined due to the

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

high overhead of manual updates and the lack of automated support embedded into development workflows.

To address these limitations, we present UMLRev, a ready-to-use tool that supports the continuous reverse engineering of UML models from feature-driven execution data. UMLRev is a working prototype integrated into the CI pipeline. It automatically generates UML class and sequence diagrams for each software release, ensuring that documentation evolves alongside the source code with minimal developer intervention. The tool was designed to reflect real-world constraints and workflows, especially those found in agile and DevOps environments.

UMLRev stands out for combining context awareness, feature orientation, and CI integration into a unified, accessible solution. Features are manually delimited by developers through test scripts—an intentional human-in-the-loop mechanism that enhances traceability and reflects how practitioners naturally reason about system behavior. This pragmatic strategy supports the tool's adoption without requiring changes to existing development practices. Thus, we define a feature as a cohesive unit of functionality that can be tested independently. Feature delimitation is performed manually by the developer through test scripts that serve as proxies for software features. This human-in-the-loop strategy enables domain-specific flexibility, allowing teams to determine what constitutes a feature in their context. All execution traces associated with the test script are considered representations of that feature during model reconstruction.

The main contributions of this paper are as follows:

- We present UMLRev, a tool that operationalizes feature-oriented and context-aware reverse engineering of UML models in CI pipelines;
- We demonstrate the integration of UMLRev into real CI workflows and describe its automation pipeline;
- We report a qualitative evaluation with 22 software professionals to assess the tool's usability, usefulness, and perceived value in agile settings.

By contributing a tangible artifact rather than a conceptual framework, this article aims to reduce the gap between model-driven engineering research and industry practice. UMLRev offers concrete benefits to practitioners, such as improved documentation accuracy and reduced onboarding time, while simultaneously opening new directions for researchers at the intersection of model-driven development and continuous software engineering.

The remainder of this article is structured as follows. Section II introduces key background concepts. Section III positions our work within the literature. Section IV details the architecture and implementation of UMLRev. Section V presents the empirical evaluation. Section VI discusses implications and limitations. Section VII concludes and outlines future work.

II. BACKGROUND

This section presents the main concepts to understand the proposed approach.

A. CONTINUOUS UML MODEL REVERSE ENGINEERING

Continuous UML model reverse engineering is the automated process of keeping UML diagrams in sync with changes made to source code. It ensures that software design documentation remains accurate and up to date throughout the development lifecycle [5], [6]. By generating UML models during each new release in a continuous integration (CI) pipeline, this approach delivers tangible benefits. Its primary goal is to maintain alignment between evolving system features and their corresponding models. Synchronizing UML documentation with source code changes helps prevent design degradation [7], [8], reduces development errors [3], [4], and clarifies the architecture of the system under construction [10]. However, most existing reverse engineering tools struggle to support this process in continuous development due to several key limitations:

- *Lack of contextual information*: Traditional tools often fail to capture and utilize the contextual information related to specific application features, leading to generic and incomplete UML models that do not accurately reflect the current state of the software.
- *Dynamic nature of software*: Continuous integration environments involve frequent changes to the software architecture. Existing tools may not be equipped to handle the rapid, ongoing updates, leading to outdated or inconsistent UML diagrams.
- *Manual effort and time constraints*: The manual effort required to update UML models continuously can be extensive and time-consuming. This makes it challenging for developers to keep UML documentation up to date alongside ongoing development.
- *Integration challenges*: Many reverse engineering tools are not seamlessly integrated into continuous integration pipelines. This lack of integration means UML models are not automatically updated with each new software release, thereby contributing to the documentation's obsolescence.

Integrating existing reverse engineering tools into a continuous integration pipeline poses several challenges, including ensuring compatibility with automated workflows. Many reverse engineering tools are not designed for automated use, making it challenging to integrate them seamlessly into pipelines that rely on automation for continuous deployment and testing. In addition, performance overhead is another concern when using automated reverse engineering tools. Each continuous integration cycle can introduce significant performance overhead, slowing down the overall build and integration process, which is designed to be rapid and efficient.

B. FEATURE-ORIENTED RESERVE ENGINEERING

Feature-oriented reverse engineering is a paradigm that aims to extract and continuously update software models based on specific functionalities—features rather than modeling the system as a monolithic whole. This perspective ensures that

design artifacts, such as UML diagrams, reflect the system's structure and behavior in terms of independently meaningful, deliverable functionalities. The result is more relevant, contextualized documentation that supports comprehension, maintenance, and evolution tasks aligned with user-visible system behavior.

In line with the principles of Feature-Oriented Software Development (FOSD) [11], features represent modular units that fulfill requirements, embody design decisions, and enable variability. In our approach, we embrace this decomposition at the modeling level, generating UML class and sequence diagrams that correspond to features rather than system-wide representations. This enables developers and architects to navigate the system structure with respect to specific user needs and product configurations, thereby strengthening traceability among features, code, and models.

A distinguishing aspect of our approach is the explicit inclusion of the *human in the loop* for defining what constitutes a *feature*. We operationalize this by associating *test scripts with features*. That is, we assume that for each relevant feature, there exists a dedicated test case or test suite that exercises its functionality. These test scripts serve as both execution drivers and semantic anchors: by running the system through feature-specific test cases, our tracer captures contextual information directly tied to a given feature's behavior. This *human-in-the-loop design* reflects real-world practice, in which developers possess tacit knowledge to scope features through their testing activities—effectively aligning reverse engineering with development intent.

This strategy shifts the feature identification problem from static code analysis to a *developer-guided dynamic analysis*. Instead of relying on heuristic or inferred mappings between features and code, our tool treats test scripts as first-class citizens for feature scoping. The benefit is twofold: (i) models reflect features as defined by those who know them best—the developers—and (ii) diagram generation remains modular, scalable, and sensitive to the actual execution paths associated with each feature. This model fosters a continuous modeling practice in agile contexts, reinforcing the role of automated documentation as a living, developer-aligned artifact.

C. CONTEXTUAL INFORMATION

When executing a test script associated with a specific software feature, it becomes possible to collect rich contextual data from the execution environment—specifically, the source code paths traversed during the feature's runtime. This execution context provides concrete evidence of how a feature manifests in the codebase, enabling the generation of software design models that are inherently sensitive to the feature's runtime behavior. Such context-aware modeling is essential to bridge the gap between static design representations and the actual dynamics of software systems.

Prior work underscores the importance of incorporating contextual information in software engineering. Briand et al. [12] argue that a comprehensive understanding of the system context is critical for informed decision-making in software design and evolution. Similarly, D'Avila et al. [13], [14] emphasize that contextual cues significantly improve developer comprehension, thereby enhancing maintainability and reducing cognitive load. Our approach (Section IV) builds upon this foundation by systematically capturing the execution context of software features during the runtime of automated test scripts. This information is used to reverse-engineer UML class and sequence diagrams that reflect not only the system's structural view but also its dynamic, feature-specific interactions.

In contrast to conventional reverse engineering techniques that generate generic or abstract models, our method ensures that the UML artifacts it produces — particularly class and sequence diagrams — capture real execution flows and dependencies that emerge from concrete feature executions. This results in more precise, semantically meaningful documentation, where each diagram faithfully represents how specific functionalities are implemented and coordinated across components. By grounding the model generation process in runtime data derived from human-defined test scripts, we align technical documentation with the developer's mental model of the system.

Maintaining relevant, up-to-date models is increasingly challenging in agile, rapidly evolving environments. Our use of contextual information directly addresses this by producing documentation that adapts alongside code changes. Furthermore, recent advances in code summarization, such as CoCoGUM [15], demonstrate that global contextual information (e.g., inter-class and intra-class dependencies) substantially improves the quality of generated summaries. This reinforces the importance of incorporating execution context in model generation, as it not only enhances documentation fidelity but also supports downstream tasks such as maintenance, onboarding, and architectural reasoning.

III. RELATED WORK

This section analyzes the related work (RW) individually (Section III-A) and then compares those works with the proposed approach based on objective criteria (Section III-B).

A. ANALYSIS OF RELATED WORK

RW1: Lano et al. [16]. This work proposes a Model-Driven Engineering (MDE) approach to facilitate software language translation by reverse-engineering source code into UML and OCL models and then forward-engineering to a target language. Their method emphasizes semantic preservation, allowing users to customize translation rules and reuse OCL components. Although the approach ensures rigorous abstraction and correctness, it lacks support for context-aware modeling and does not consider feature-level information. Moreover, it neither integrates with CI pipelines nor supports continuous UML updates. While it introduces a semantically rich transformation process, the absence of functional tooling, empirical validation, and CI integration limits its applicability to evolving systems.

Unlike UMLRev, which automatically extracts contextual feature-based execution data and continuously updates UML models, RW1 focuses on language interoperability rather than on maintaining design models in fast-paced delivery environments.

RW2: Edan et al. [17]. This study examines a multi-module software solution, with a focus on the reverse engineering of backend APIs to repair and enhance existing systems. The authors organize the process into stages, including documentation arrangement, analysis, reverse engineering, and development. Although they produce UML artifacts and include stakeholder interviews, the approach lacks automation and does not integrate with modern CI/CD workflows. There is no provision for continuously updated UML models or for managing feature-oriented concerns. While the method provides valuable design documentation, it lacks context-aware mechanisms or tool support to handle evolving codebases. UMLRev, in contrast, addresses these limitations by embedding feature-level awareness and automation directly into the CI pipeline, ensuring the timely evolution of class and sequence diagrams aligned with every release.

RW3: Pereira et al. [18]. TRACEM extends an MDE framework by incorporating both static and dynamic analysis to support software modernization. The initiative emphasizes enriching reverse engineering with runtime behavioral insights and presents a case study demonstrating the value of integrating data sources. However, while TRACEM introduces important refinements to model extraction quality, it lacks integration with continuous development environments and omits feature orientation entirely. There is no indication of automated model updates or scalable CI support. Although TRACEM and UMLRev both aim to improve reverse engineering, only UMLRev operationalizes this in CI environments, making it better suited for agile, fast-evolving projects.

RW4: Rosca et al. [19]. This work conducts a comparative study on the round-trip engineering (RTE) capabilities of Papyrus, Modelio, and Visual Paradigm. The analysis evaluates whether these tools can maintain consistency between models and code through successive development iterations. Although the study provides insight into tooling support for model-code synchronization, it does not propose a new method or tool. Furthermore, none of the evaluated tools offer context-sensitive modeling or CI integration. While the study underscores the relevance of RTE, it does not explicitly address continuous reverse engineering or feature-aware updates. UMLRev extends beyond traditional RTE tools by embedding semantic and contextual understanding into the CI process, enabling the creation of meaningful, continuously evolving UML models.

RW5: Src2MoF (2019) [20]. The Src2MoF approach focuses on improving MDRE by generating UML diagrams from Java source code. While effective for static code analysis, the method does not handle runtime behavior or evolving feature-level context. It lacks CI integration, does not update diagrams across releases, and provides no mechanism to maintain model relevance in agile workflows. Although it makes valuable contributions to initial model recovery, Src2MoF falls short in addressing the continuous, automated needs of modern development pipelines. In contrast, UMLRev addresses these shortcomings by embedding reverse engineering into CI pipelines and aligning diagram updates with the execution context of features.

RW6: Baidada et al. [21]. This work presents a method for extracting UML2 sequence diagrams from execution traces by analyzing Colored Petri Nets generated from multiple use case scenarios. The process allows high-level behavioral abstractions and improves model comprehensibility. However, the approach does not scale to continuous environments or support CI/CD workflows. It also lacks automation for large systems and omits context- and feature-level modeling. Unlike UMLRev, which systematically generates and updates both class and sequence diagrams based on contextual execution, this approach remains confined to batch processing of trace logs.

RW7: Braude [22]. Braude introduces a discipline for embedding UML class diagrams incrementally within source code, aiming to support continuous agile development. The method leverages primary functions and postconditions to construct models aligned with specific requirements, enabling the creation of traceable UML elements. While the technique embraces incremental modeling, it is manual, tool-dependent, and not integrated with CI workflows. It does not support feature orientation, runtime context awareness, or automated diagram updates. UMLRev differentiates itself by automating these processes, ensuring UML artifacts evolve with each code integration cycle.

RW8: Labiche et al. [23]. This study proposes a hybrid static and dynamic analysis technique to reverse engineer scenario diagrams with reduced reliance on bytecode instrumentation. The study contributes a dedicated algorithm to improve object-oriented reverse engineering. However, it operates as a standalone method without integration into CI workflows or support for feature-based documentation. While effective in capturing behavior at runtime, the approach lacks the scalability and automation required for continuous updates. UMLRev addresses these gaps by enabling context-aware diagram generation embedded in modern delivery pipelines.

RW9: Vinita [24]. It outlines an algorithm for generating UML class diagrams from object-oriented source code, highlighting its applicability to legacy systems. While it successfully illustrates MDRE techniques, the method is limited to static recovery and does not account for runtime behavior or context. There is no mention of CI integration, automation, or diagram synchronization across releases. Although the approach lays the foundation for model extraction, UMLRev advances this by ensuring the continuous, context-sensitive evolution of UML artifacts aligned with software features.

RW10: Briand et al. [25]. It explores the use of dynamic analysis in combination with aspect-oriented programming (AOP) to reverse-engineer distributed Java systems. Their technique captures message synchronization between components using AspectJ and introduces novel mechanisms for analyzing dynamic behavior. Although AOP enhances observability, the tool lacks full automation, CI integration, and practical guidance for continuous use. UMLRev builds on these foundations by embedding automated, feature-oriented instrumentation into CI workflows, ensuring consistent, up-to-date UML representations of software behavior.

RW11: Merdes et al. [26]. It introduces a tool to reconstruct UML sequence diagrams from Java program executions. The system addresses key implementation concerns, including data collection, visualization, and editing. While functional, it lacks support for context-aware modeling and does not scale for continuous updates or integration with version control and CI tools. Although the study provides valuable insights into reverse-engineering tooling, its scope is limited to single-instance executions. UMLRev surpasses this by automating model evolution through continuous monitoring of feature executions and software releases.

RW12: Antoniol et al. [27]. WANDA is a reverse engineering tool designed to recover architectural and UML documentation for web applications. It leverages both static and dynamic analyses, extending Conallen diagrams to include runtime behaviors. Though WANDA supports customization and extensibility, it is not context-aware and lacks CI integration or support for continuous updates. While useful for profiling and maintenance, it omits feature-level granularity and developer-centric evaluations. UMLRev complements WANDA's runtime emphasis by providing scalable automation, CI compatibility, and a focus on feature-driven modeling.

RW13: Stroulia et al. [28]. It emphasizes the importance of dynamic analysis in understanding undocumented legacy systems. Their work surveys standard reverse-engineering practices and advocates deeper behavioral analysis of object-oriented systems. While they frame essential challenges and opportunities, the work remains conceptual and does not offer tooling or CI-ready solutions. UMLRev advances this vision by operationalizing behavioral insights into automated, up-to-date UML models embedded in CI/CD pipelines, addressing the practical needs of modern software development.

B. COMPARATIVE ANALYSIS AND RESEARCH GAP

1) ANALYTICAL SYNTHESIS AND KEY OBSERVATIONS

To ensure an unbiased and rigorous comparison, we evaluated the 13 related studies using objective and verifiable criteria aligned with state-of-the-art reverse engineering practices. These criteria—context awareness, feature orientation, CI integration, automation, continuous model updates, and qualitative evaluation—were carefully selected to reflect the capabilities expected of modern reverse engineering tools operating in dynamic software environments. This comparative framework avoids subjective judgment by applying binary or traceable classifications. It follows established practices observed in previously published and validated research (e.g., [29], [30]). The comparative table (Table 1) offers a transparent lens through which the functional and methodological gaps in the literature become evident. We list some key observations:

- *Context awareness*: Few approaches effectively capture contextual information during model recovery. Although RW3, RW10, and RW13 incorporate elements of static or dynamic context, none embed this information within feature execution flows or CI-integrated pipelines. UMLRev distinguishes itself by proactively capturing feature-centric context derived from operational environments.
- *Feature orientation*: Except for RW10, which leverages aspect-oriented constructs to isolate cross-cutting concerns, the reviewed studies do not adopt feature orientation as a design principle in their reverse engineering strategies. UMLRev introduces an explicit feature-oriented perspective, allowing traceability between software features and their architectural representations.
- *CI integration*: Continuous integration support is notably absent across all related studies. UMLRev uniquely integrates with CI/CD environments (e.g., GitHub Actions, GitLab CI), enabling real-time reverse engineering as part of the software delivery lifecycle.
- *Automation and continuous model updates*: While several approaches offer partial automation (e.g., RW1, RW5, RW6, RW8, RW11), they operate as standalone tools without continuous synchronization. UMLRev advances this by enabling fully automated model recovery and constant updates, reducing architectural drift and improving maintainability.
- *Qualitative evaluation*: Only UMLRev and, to a limited extent, RW2 incorporate qualitative feedback from practitioners. However, RW2 provides minimal detail and lacks interpretive depth. UMLRev, in contrast, provides structured empirical insights from interviews and observational studies, thereby validating both usability and real-world impact.

2) RESEARCH OPPORTUNITIES AND ARTICLE ROADMAP

The comparative analysis reveals some research opportunities that remain insufficiently explored in the current literature and that the proposed article explicitly addresses. We identify the main research gaps and specify the corresponding sections in our article where each gap is thoroughly addressed:

1) *Bridging context awareness with CI pipelines*: No existing work offers a context-aware approach that integrates into continuous development environments. This gap is addressed in Section IV (Proposed Approach), where UMLRev's architecture enables

TABLE 1. Comparison of related works with UMLRev.

Related Work	Context Awareness	Feature-Oriented	Integrated to CI cycle	Complete Automation	Continuous UML Updates	Qualitative Evaluation
UMLRev (Proposed)	●	●	●	●	●	●
RW1: Lano *et al.* (2024) [16]	○	○	○	◐	○	○
RW2: Edan *et al.* (2022) [17]	○	○	○	○	○	◐
RW3: Pereira *et al.* (2022) [18]	◐	○	○	○	○	○
RW4: Roşca *et al.* (2021) [19]	○	○	○	○	○	○
RW5: Src2MoF (2019) [20]	○	○	○	◐	○	○
RW6: Baidada *et al.* (2019) [21]	○	○	○	◐	○	○
RW7: Braude (2017) [22]	○	○	○	○	○	○
RW8: Labiche *et al.* (2013) [23]	○	○	○	◐	○	○
RW9: Vinita (2008) [24]	○	○	○	○	○	○
RW10: Briand *et al.* (2006) [25]	◐	●	○	○	○	○
RW11: Merdes *et al.* (2006) [26]	○	○	○	◐	○	○
RW12: Antoniol *et al.* (2004) [27]	○	○	○	◐	○	○
RW13: Stroulia *et al.* (2002) [28]	◐	○	○	○	○	○

Legend: (●) *Meets* (◐) *Partially meets* (○) *Does not meet*

automatic extraction of context from feature traces during builds and deployments.

2) *Incorporating feature orientation into reverse engineering*: The lack of feature-to-model mapping strategies in prior works highlights the need for a feature-oriented reverse engineering paradigm. UMLRev fills this void by supporting traceability from software features to UML elements, as detailed in Sections IV and VI.
3) *Enabling automation with continuous model evolution*: Most tools perform one-shot extractions. UMLRev pioneers a continuous reverse-engineering loop, tightly coupled with CI events. This is operationalized through the CI tool described in Sections IV and V, which showcases its feasibility and integration into realistic workflows.
4) *Qualitative evaluation*: A major shortcoming in previous work is the absence of empirical evaluation grounded in developer experiences. UMLRev includes a qualitative study, reported in Section V, capturing insights from developers using the tool in CI pipelines.
5) *Enhancing practical relevance through tool support*: Previous studies rarely discuss feedback mechanisms or their implications for practice. UMLRev is developed with iterative user feedback and validated through field deployment. These implications are examined in depth in Section VI, connecting practical use with scientific contributions.

Together, these research opportunities highlight the originality of the proposed approach, guiding the article's structure and ensuring that each identified gap is substantively explored and addressed.

IV. THE PROPOSED APPROACH

This section introduces UMLRev (UML Reverse), a context-aware, feature-oriented approach that supports the continuous reverse engineering of UML models. Section IV-A gives an overview of our proposed approach. Next, Section IV-B describes the proposed component-based architecture. Section IV-C introduces the proposed context model. Then, Section IV-D presents the main algorithms. Section IV-E introduces a realistic usage scenario where the UMLRev generates UML class and sequence diagrams at an implementation level. Finally, Section IV-F discusses details about implementation aspects.

A. OVERVIEW OF THE PROPOSED APPROACH

Figure 1 presents an overview of the proposed continuous reverse engineering process that UMLRev supports. This process leverages modern tools commonly utilized within continuous delivery, including GitHub and continuous integration tools (such as Travis[1] and GitLab[2]) to generate UML models. We describe each step of this process as follows:

- **Step 1: Pull request submission.** It defines and sends users' pull requests, enabling them to notify others of the modifications they have pushed to a specific branch in a repository hosted on GitHub. Users discuss and review potential changes with collaborators, then add follow-up commits before merging their changes into the base branch.
- **Step 2: Approve the pull request.** The UMLRev's user approves the pull request on GitHub, and then GitLab (or Travis) runs integration tests.
- **Step 3: Continuous configuration tool.** This step focuses on compiling the newly submitted source code, running automated tests, and triggering the tool developed for model generation. The UMLRev notifies the user if the compilation or automated tests fail.
- **Step 4: Model generation.** After the GitLab (or Travis) completes software compilation and automated testing, the UML Rev tool runs on the GitLab (or Travis) CI server itself. The UMLRev re-executes the integration tests to capture context information about the execution of the application features. It captures information from the code snippets executed to run a feature, tracing the execution flow using aspect-oriented programming. Then, the tool generates the UML class and sequence diagrams for each application feature,

[1]Travis CI: https://www.travis-ci.com/
[2]GitLab: https://about.gitlab.com/

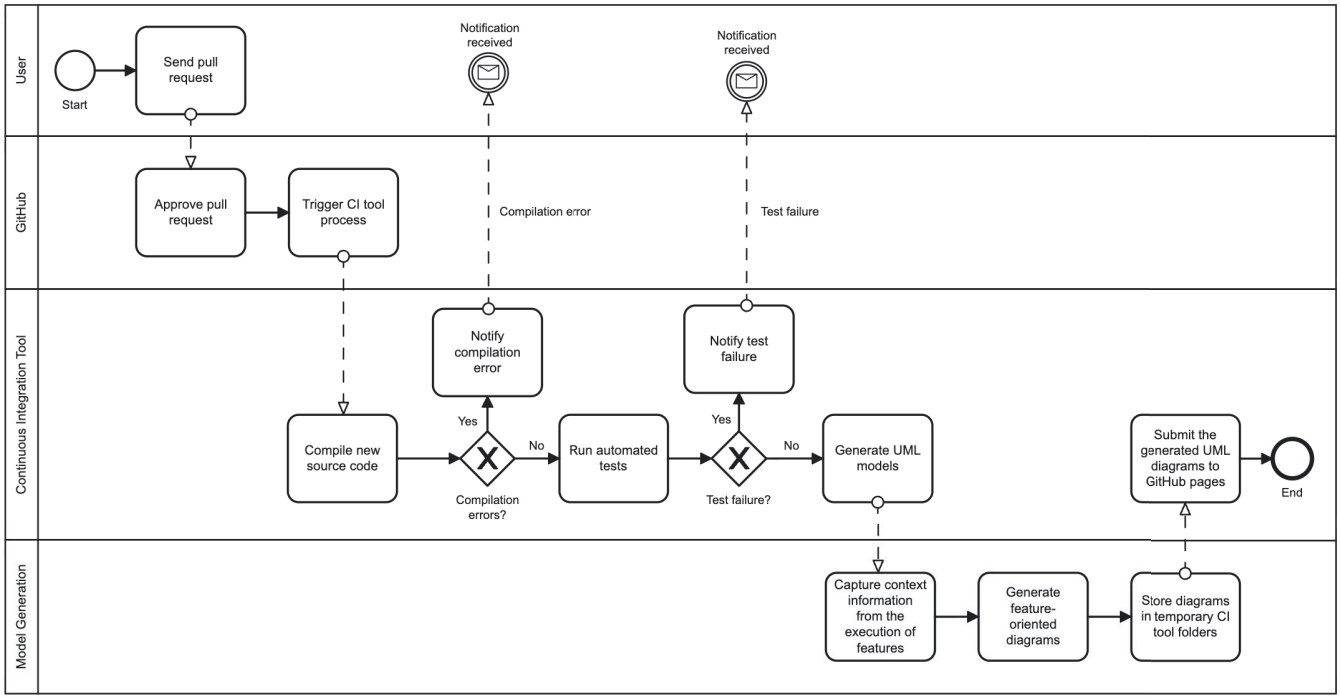


FIGURE 1. Workflow showing CI pipeline triggered by a GitHub pull request, compiling code, running tests, generating UML models, and publishing diagrams when no errors or failures occur.

taking into account the captured information. Each feature is extracted based on the code snippets that were executed. Test scripts can be created to test the system features. When a test script is executed, the activated code snippets are captured. For example, 20 UML diagrams will be generated if 10 application features are tested, i.e., 10 class diagrams and 10 sequence diagrams. These diagrams, produced by running UMLRev, are stored in a local, temporary folder on the GitLab (or Travis) CI server. Finally, they are sent to GitHub Pages via a script configured for this purpose, executed on the GitLab (or Travis) CI server.

B. COMPONENT-BASED ARCHITECTURE

Figure 2 illustrates the component-based architecture designed to support the implementation of the proposed UMLRev approach. Each component is responsible for a specific phase of the process described in Figure 1, emphasizing abstraction and modularity. We detail the responsibilities and interactions of each component, highlighting their roles in a loosely coupled, service-oriented architecture:

- **Tracer:** The `Tracer` component is implemented using aspect-oriented mechanisms to monitor the execution of the target software during CI cycles. Its primary function is to record relevant behavioral events and persist them through the `RepositoryRecorder` interface. The traced information captures the contextual execution path associated with developer-defined test scripts. This component ensures the system remains non-intrusive by intercepting runtime events transparently, without modifying the underlying business logic.
- **Repository:** The `Repository` acts as a centralized persistence layer responsible for storing all collected trace data and intermediate model representations. It is accessed through the `RepositoryProvider` and `RepositoryRecorder` interfaces, which abstract the storage mechanism (e.g., in-memory, database, or file system). In our current prototype, an in-memory implementation is adopted to prioritize performance and reduce complexity. This component ensures data decoupling, promoting modularity and reusability across the pipeline.
- **Diagram Engine:** The `Diagram Engine` is the core orchestrator responsible for coordinating the diagram generation workflow. It retrieves processed information from the `Information Model`, invokes the appropriate `Diagram Strategy`, and interacts with the underlying modeling framework through the `Metamodel Adapter`. The engine abstracts low-level concerns and enforces a clean separation between orchestration logic and transformation mechanics.
- **Diagram Strategy:** This component encapsulates multiple strategies for generating different types of UML diagrams (e.g., class and sequence diagrams). Each strategy operates on structured information provided by the `Information Model` and relies on the `Metamodel Adapter` to instantiate UML-compliant elements. The architecture is open for extension, allowing new diagram strategies to be added without

altering the existing pipeline, thus ensuring scalability and maintainability.

- **Information Model:** The `Information Model` defines the internal representation used to model the extracted execution context. It translates raw trace data from the `Repository` into a structured format containing method calls, object instances, and their relationships. This abstraction layer supports semantic enrichment and serves as input for diagram strategies. It exposes the `InformationFactory` interface to facilitate data transformation and consumption.
- **Metamodel Adapter:** Acting as a bridge to external modeling frameworks, the `Metamodel Adapter` ensures that all elements generated by the diagram strategies conform to a standard UML metamodel. In our implementation, we utilize the Eclipse UML2 framework.[3] This adapter provides flexibility, allowing integration with other metamodels or diagramming libraries without modifying upstream components.

The architecture promotes separation of concerns and adheres to component-based software engineering principles. Interfaces (e.g., `InformationFactory`, `RepositoryProvider`) are explicitly defined to enable independent evolution of components, facilitating testability and modular deployment within CI pipelines.

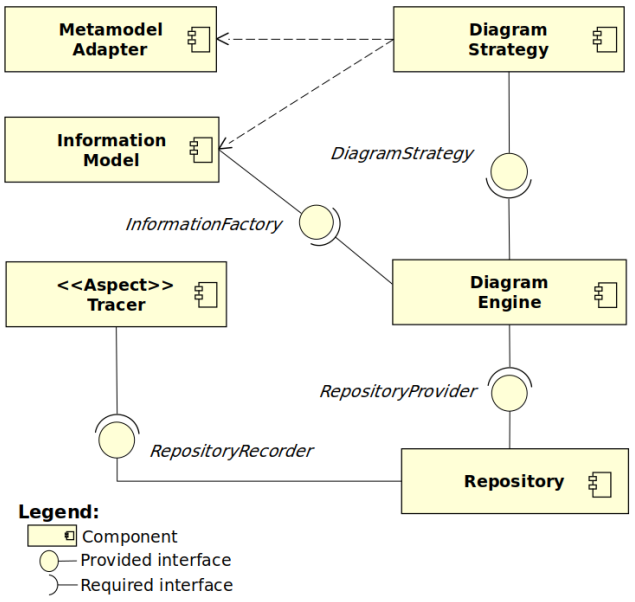


FIGURE 2. Component-based architecture of the UMLRev approach showing interactions among metamodel Adapter, Information model, Tracer, Diagram strategy, Diagram engine, and repository via provided and required interfaces for diagram generation and repository recording.

1) FROM COMPONENT ABSTRACTION TO CLASS REALIZATION

The class diagram presented in Figure 3 shows a realization of the abstract architecture defined in the component-based model (Figure 2). Each package in the class diagram corresponds directly to a component in the architecture, preserving the system's modularity while specifying implementation details. For instance, the `tracer` package instantiates the `Tracer` aspect, now represented as a class with methods `start()` and `stop()`, and an interface `RepositoryRecorder`, faithfully capturing the role of recording observed events during execution. This component collaborates with the `repository` package, where `RepositoryInMemory` implements both the recorder and provider roles, enabling storage and retrieval of information. The interfaces `RepositoryRecorder` and `RepositoryProvider` decouple the responsibilities for recording and retrieval, as depicted in the component-level interfaces.

2) ENCAPSULATION OF DIAGRAM GENERATION LOGIC

The packages `diagram`, `strategy`, and `information-model` work in tandem to implement the logic for UML diagram generation, aligning precisely with their component counterparts. The `diagram` package implements the `Diagram Engine` component, in which the `DiagramEngine` class invokes strategies to generate diagrams based on the repository's extracted information. The interface `DiagramStrategy` enables polymorphism, allowing dynamic delegation to specific generators such as `ClassDiagram` or `SequenceDiagram`, defined in the `strategy` package. Within `strategy`, adapter classes (`AdapterClassToUml2` and `AdapterSequence-ToUml2`) bridge the gap between high-level logic and low-level UML2 constructs, as anticipated by the `Metamodel Adapter` component. Meanwhile, the `information-model` package implements the data abstraction layer for diagram construction, including atomic units such as `IClass`, `IMethod`, and `IReturn`, which are managed by `InformationFactoryImpl`. This modular, layered implementation confirms that the class diagram operationalizes the responsibilities and interfaces identified in the component diagram, ensuring conceptual continuity and architectural consistency throughout the design.

C. CONTEXT MODEL

Figure 4 shows the conceptual model of the *UMLRev contextual model* in which its key entities and relationships are expressed. The proposed context model enables automated extraction, storage, and organization of execution data from use-case scenarios, thereby facilitating the generation of UML class and sequence diagrams. It systematically captures runtime information and maintains traceability between use case executions and the generated UML models. Execution data is collected when a *UseCaseScenario* is triggered.

Each scenario is associated with an `Actor`, representing the user or system component interacting with the application, and a `SystemEvent`, which records key system interactions such as API calls, UI events, or method executions. These events are logged with timestamps, allowing for

[3]UML2: http://wiki.eclipse.org/MDT/UML2/

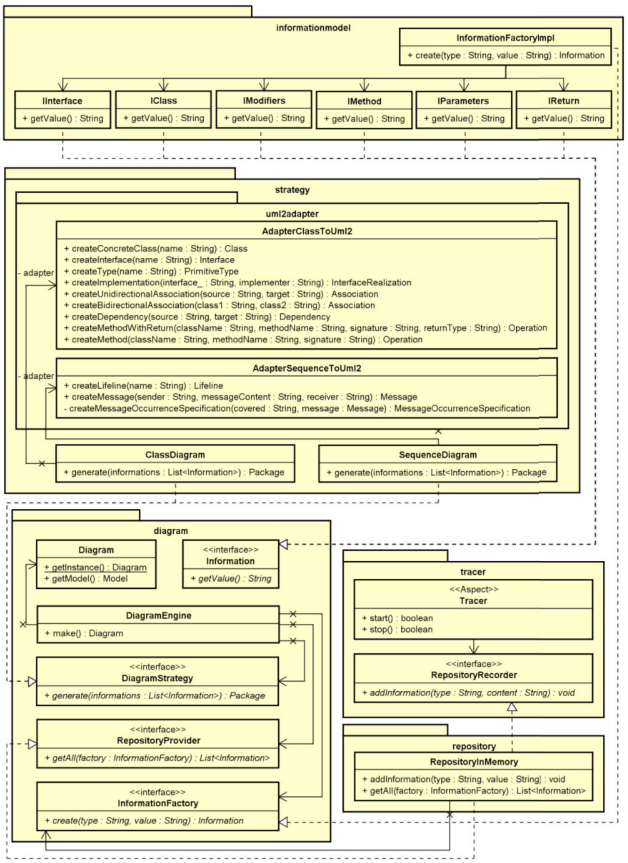


FIGURE 3. UML class diagram detailing components for diagram generation, strategy adapters, information modeling, tracing, and repository management, showing classes, interfaces, attributes, operations, and their relationships.

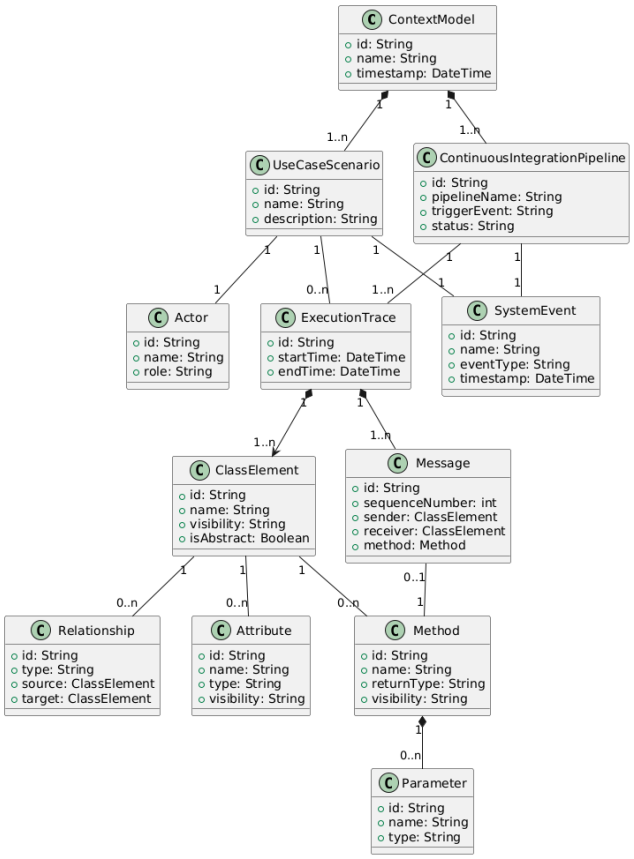


FIGURE 4. UML class diagram of a context model linking use case scenarios, continuous integration pipelines, execution traces, class elements, messages, and related entities with attributes and relationships for software process representation.

accurate traceability. To track detailed execution behavior, an `ExecutionTrace` is created to log the start and end times of the scenario's execution. Each trace records method calls and object interactions, which are later used to reverse-engineer UML models.

The `ExecutionTrace` captures class structures dynamically instantiated during execution. These structures are represented as `ClassElement` entities containing `Attributes` and `Methods` extracted from the execution environment. Each method may have multiple `Parameters`, and `Relationships` between class elements are identified based on object interactions. This structural information serves as the basis for generating UML class diagrams that reflect the system's design as observed during execution.

The *Message* entity records method calls exchanged between classes to construct UML sequence diagrams. Each Message contains a `sequenceNumber` that defines the execution order, along with a sender and receiver representing the interacting `ClassElements`. This enables an accurate representation of dynamic behavior, showing method invocation flows within the system. To support automated, continuous UML model updates, the `ContinuousIntegrationPipeline` entity links execution traces to CI/CD pipelines such as GitHub Actions, GitLab CI, or Travis CI. Each pipeline execution captures `SystemEvents` and `ExecutionTraces`, ensuring that UML models reflect the latest software changes.

The proposed context model integrates context-awareness (capturing execution traces and system events), reverse engineering (extracting structure and behavior from runtime execution), and support for continuous integration (updating models dynamically). By maintaining explicit relationships between use case scenarios, execution traces, and UML representations, this model ensures the creation of accurate, automated, and continuously updated UML class and sequence diagrams in an evolving software system.

D. ALGORITHMS

The following high-level steps provide a simplified overview of how the proposed approach observes the feature execution, collects the execution flow, and uses this data to generate UML class and sequence diagrams. This separation between data collection and diagram generation enhances modularity and extensibility, facilitating the incorporation of new strategies for collecting and generating developer-facing diagrams.

```
aspect Tracer {
    // Captures method call join points
    pointcut methodCall(): call(* *.*(..)) && immune();
    // Captures method execution join points
    pointcut methodExecution(): execution(* *.*(..)) &&
        immune();
    // Captures constructor call join points
    pointcut constructorCall(): call(*.new(..)) && immune
        ();
    // Captures constructor execution join points
    pointcut constructorExecution(): execution(*.new(..))
        && immune();
}
```

Listing 1. Source code showing the main pointcuts of the Tracer component.

Step 1: Execution Monitoring. Use the `Tracer` component to monitor software execution, which uses AspectJ to define and configure pointcuts. These pointcuts specify points of interest in the program's execution, such as method calls and method executions, constructor calls and constructor executions. They are specifically designed to exclude points internal to the aspect, avoiding unwanted recursion and ensuring that all relevant interactions are recorded without interference from the monitoring mechanism. The use of pointcuts and advice in AspectJ effectively captures these relevant events. Listing 1 provides an AspectJ code snippet that demonstrates how the Tracer component is implemented to achieve this functionality.

Step 2: Data collection. The information captured by the pointcuts during the execution is stored in a data structure, such as a repository, and is later accessed by the `DiagramEngine` for diagram generation. This modular approach ensures that data collection and processing are organized and efficient, enabling a streamlined workflow from data capture to diagram creation.

Step 3: Diagram generation. Use the `DiagramEngine` component to process the collected data. For each available diagram strategy (`DiagramStrategy`), call the `generate() method` (Listing 2), passing the collected data. Each `DiagramStrategy` implements the generation of a specific type of diagram. The *make()* method in the `DiagramEngine` (Listing 3) serves as the central hub for the diagram generation process. It systematically processes each implemented diagram strategy, applying it individually to generate the final diagrams. This structured approach ensures that all data are used effectively to produce accurate and detailed UML diagrams in accordance with the predefined strategies within the `DiagramEngine`.

```
class DiagramEngine {
    private List<DiagramStrategy> strategies;
    private Repository repository;
    private Factory factory;
    public Diagram make() {
        for (DiagramStrategy diagram : strategies) {
            diagram.generate(repository.getAll(factory));
        }
        return Diagram.getInstance();
    }
}
```

Listing 2. A source code showing the main component to generate UML diagrams.

Each strategy, implemented in the `ClassDiagram` and `SequenceDiagram` classes, processes the collected data to generate a specific diagram type. Listing 3 outlines the strategies for generating class diagrams.

```
class ClassDiagram implements DiagramStrategy {

    private AdapterClassToUml2 adapter;

    @Override
    public Package generate(List<Information> information
        ) {
        if (information != null && !information.isEmpty()
            ) {
            generateClasses(information);
            generateInterfaces(information);
            generateImplementations(information);
            generateAssociations(information);
            generateDependencies(information);
            generateTypes(information);
            generateMethods(information);
        }
        return adapter.getPackage();
    }
}
```

Listing 3. A source code showing the ClassDiagram component.

```
public class SequenceDiagram implements DiagramStrategy {

    private AdapterSequenceToUml2 adapter;

    @Override
    public Package generate(List<Information> information
        ) {
        if (information != null && !information.isEmpty()
            ) {
            information = addFooterAndHeader(information)
                ;
            listLifelines(information);
            arrangeMessages(information);
            generateLifelines();
            generateMessages();
        }
        return adapter.getPackage();
    }
}
```

Listing 4. A source code showing the sequence diagram strategy.

E. USAGE SCENARIO

This section presents a practical use case that demonstrates how the UMLRev tool leverages its capabilities in a realistic application context. Specifically, we examine the automated generation of UML diagrams for a software feature called Tell Story. This scenario illustrates how UMLRev captures dynamic interactions within a software system to produce detailed class and sequence diagrams. Figure 5 shows the UML class diagram generated for the *Tell Story* feature.

This feature starts with the execution of the *Story.tellStory()* method. Listing 5 presents the source of the usage scenario. Figure 6 depicts a UML sequence diagram generated from the execution flow of the methods. This diagram illustrates the classes (and their corresponding objects) that comprise the feature, their respective messages, and the sequence in which they occur. UMLRev can produce class and sequence diagrams, exclusively considering the functionality being analyzed. The goal is to capture contextual information about objects instantiated and invoked at

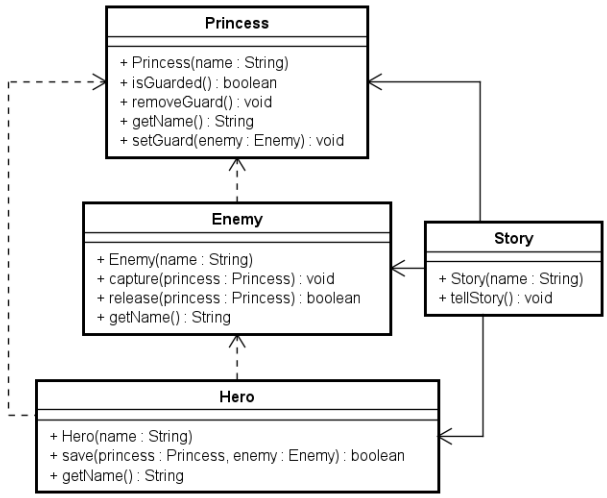


FIGURE 5. UML class diagram showing relationships among Story, Hero, Enemy, and Princess classes, with attributes, methods, and associations representing interactions in a narrative scenario.

runtime, tracing the system's execution path to a specific feature. Each diagram depicts the behavior and scope of the system for the feature under analysis and may contain information that differs from that obtained from static analysis.

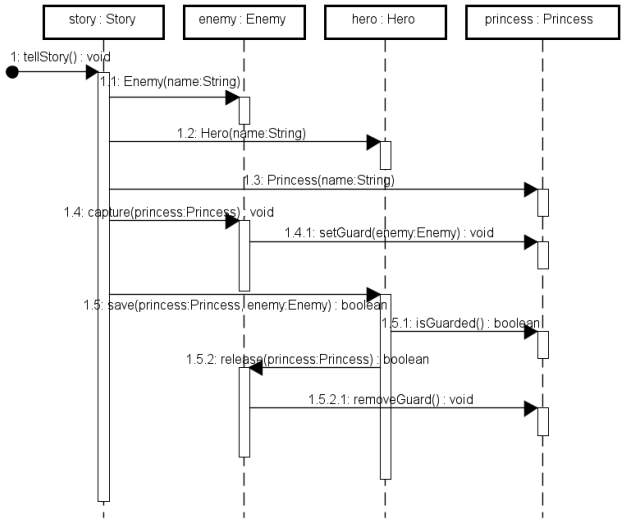


FIGURE 6. UML sequence diagram showing interactions among Story, Enemy, Hero, and Princess objects, including object creation, capture, guarding, saving, and releasing actions with nested method calls.

F. IMPLEMENTATION ASPECTS

The essence of our approach lies in its capacity for reverse engineering based on context-aware and feature-oriented concepts. Aspect orientation, powered by AspectJ, is a game-changer. It captures crucial contextual information about the application's feature execution, relieving developers of the tedious task of manual analysis. The instrumentalization

```
1  public class Main {
2      public static void main(String[] args) {
3          Story story = new Story("Epic Rescue");
4          story.tellStory();
5      }
6  }
7
8  class Story {
9      private Hero hero;
10     private Enemy enemy;
11     private Princess princess;
12     public Story(String title) { /* constructor logic
           here */ }
13     public void tellStory() {
14         this.hero = new Hero("Knight Arthur");
15         this.enemy = new Enemy("Dark Sorcerer");
16         this.princess = new Princess("Princess Amelia");
17         enemy.capture(princess);
18         hero.save(princess, enemy);
19     }
20 }
21 class Princess {
22     private Enemy guardedBy;
23
24     public void setGuard(Enemy enemy) {
25         this.guardedBy = enemy;
26     }
27     public void removeGuard() {
28         this.guardedBy = null;
29     }
30     public boolean isGuarded() {
31         return this.guardedBy != null;
32     }
33 }
34 class Enemy {
35     public void capture(Princess princess) {
36         princess.setGuard(this);
37     }
38     public void release(Princess princess) {
39         princess.removeGuard();
40     }
41 }
42 class Hero {
43     public void save(Princess princess, Enemy enemy) {
44         if (princess.isGuarded()) {
45             enemy.release(princess);
46         }
47     }
48 }
```

Listing 5. A source code showing classes used in our case study.

of the source code using aspect-oriented programming preserves critical information, such as class and method names, which may be lost when examining binary code. Dependency management is facilitated by Maven,[4] chosen for its straightforward configuration. The UMLRev source code is available on GitHub.[5]

Additionally, the generated diagrams must be stored in a location accessible to the continuous integration tool on the server. The UML diagrams were generated using the Eclipse UML2,[6] which is an EMF-based implementation of the UML metamodel [31] for the Eclipse platform. The continuous integration pipeline is configured using four distinct tools: Travis CI, GitHub,[7] GitLab, and GitHub Pages.[8] We used Travis CI and GitLab, distributed continuous integration services, to build and test software projects hosted on GitHub. We used Travis and GitLab for several reasons, including

[4]Maven: https://maven.apache.org
[5]UMLRev Repository: https://github.com/cristianoccf/UMLRev
[6]Eclipse MDT UML2: https://projects.eclipse.org/projects/modeling.mdt.uml2
[7]GitHub: https://github.com
[8]GitHub Pages: https://pages.github.com/

the absence of the need to install a dedicated server, ease of configuration, seamless integration with GitHub, and the ability to execute scripts, thereby delivering the necessary versatility.

V. EVALUATION

This section describes how we conducted a qualitative, human-based empirical evaluation, following well-established guidelines [32], [33], to assess potential end users' perceptions of the UMLRev approach's usefulness, benefits, and intention to use.

A. PARTICIPANT PROFILE ANALYSIS

1) SCREENING

The participants for this study were selected based on strict criteria to ensure a rigorous and representative sample. Each participant was required to confirm their experience with software development, reverse engineering, software modeling, and continuous integration to be eligible for the study. We used convenience sampling [32] to select the participants. Each participant, including industry professionals, graduate students, and undergraduates, was invited by email.

We conducted the study with 22 participants, comprising 12 professionals from Brazilian companies and 10 students with professional experience. Participants received the same training on the key features of the proposed approach. The professionals held Master's and Bachelor's degrees (or equivalent) and had considerable knowledge of software modeling, reverse engineering, and practical programming. They were part of a postgraduate program in Applied Computing at a Brazilian university. The graduating students participated in our evaluation, comprising subjects with diverse backgrounds and levels of expertise.

These students had a rigorous background in software modeling and practical software development knowledge gained through professional experience on real-world enterprise projects. Before answering the formulated questionnaire (described in Section V-B), the participants received training on the proposed approach through practical continuous reverse engineering scenarios using the developed tool. We took great care to ensure that each participant thoroughly understood the proposed approach, instilling confidence in the study's results.

2) PARTICIPANTS' PROFILE

Table 2 describes the participants' profile, reporting their main characteristics, including age, education, level of education, academic learning time, overall professional experience with software modeling and development, current position, and current position in years. A total of 22 participants responded to the formulated questionnaire.

Education & Undergraduate Course. The majority (54.5%) had a complete graduate degree, while 45.5% completed some postgraduate studies in computing. 50% had a Master's degree in the field of computing. Regardless of their level of education, all participants were professionals with development experience gained from real-world projects involving rapidly changing enterprise applications. All participants took an undergraduate computing course, including Computer Science (40.9%), Systems Analysis (40.9%), Computing Engineering (9.1%), Information Systems (4.5%), and Systems for the Internet (4.5%). This data highlights the participant's robust academic foundation, complementing their practical experience.

Academic learning time & Age. We also considered academic learning time and age as key factors in constructing our sample. The first refers to the duration of our participants' active engagement in learning relevant academic content. An empirical study evaluating an approach to reverse engineering UML models must include participants with a solid academic background for two main reasons. First, their advanced understanding of software engineering principles enables them to critically assess the technical aspects and the approach's effectiveness.

Second, their familiarity with UML and reverse-engineering methodologies enables more informed, nuanced feedback, thereby enhancing the study's validity and reliability. Over 54% indicated having seven years (or more) of active learning in undergraduate and graduate courses. 63.6% of participants were between 21 and 30 years old, while the remainder were between 31 and 45 years old, yielding a sample with an age range of 14 years. That is, the participants' ages are distributed rather than concentrated. Therefore, the recruited participants had diverse life experiences.

Overall experience. Over 58% indicated having seven years (or more) of professional experience, of which 36.4% had more than eight years of practical experience in realistic software development scenarios in Brazilian companies. The data indicate a lower concentration (36.3%) at the experience level of up to 4 years. The sample does not include only experienced participants, which could bias the results. Instead, it considers people with varying levels of experience, thereby enabling the practical usefulness of the proposed approach to be assessed from a broader range of perspectives.

B. QUESTIONNAIRE

We collected data using an online questionnaire created in Google Forms. Table 3 details the questionnaire applied to evaluate the UMLRev qualitatively. The questionnaire's Likert scale ranges from *Completely Disagree* to *Completely Agree*. It was designed based on (1) well-established guidelines for formulating questionnaire [32], [34], (2) known technology acceptance models [35], (3) findings on the adoption of UML modeling in industry [36], and (4) software modeling in continuous integration [9]. Furthermore, the questionnaire could be administered quickly and easily to collect data from participants across geographically diverse locations.

Questionnaires using Likert-scale items employed a structured, quantitative method to assess perceived effectiveness,

TABLE 2. Our collected data considering demographic information gathered from 22 participants.

Characteristic	Response	Quantity	Percentage
Education	Graduation	10	45.5%
	Master's degree	11	50.0%
	Postgraduate	1	4.5%
Undergraduate course	Computer Science	9	40.9%
	Systems Analysis	9	40.9%
	Computer Engineering	2	9.1%
	Information Systems	1	4.5%
	Systems for Internet	1	4.5%
Academic learning time	< 2 years	1	4.5%
	2-4 years	1	4.5%
	5-6 years	8	36.4%
	7-8 years	6	27.3%
	> 8 years	6	27.3%
Age	21-25 years	7	31.8%
	26-30 years	7	31.8%
	31-35 years	3	13.6%
	36-40 years	3	13.6%
	41-45 years	2	9.1%
Overall experience	< years	5	22.7%
	2-4 years	3	13.6%
	5-6 years	1	4.5%
	7-8 years	5	22.7%
	> 8 years	8	36.4%
Current position	Programmer	7	31.8%
	Systems analyst	4	18.2%
	Software Architect	4	18.2%
	Trainee	3	13.6%
	Manager	1	4.5%
	Researcher	1	4.5%
	Technology Director	1	4.5%
	Lecturer	1	4.5%

usefulness, and attitudes towards the proposed approach. This provided valuable insights into stakeholders' perspectives and facilitated data analysis and interpretation. Our questionnaire evaluated essential issues based on the participants' perceptions regarding continuous reverse engineering, diagram automation, the usefulness of feature-oriented diagrams, and the practical application of UML models. It detailed the interaction between traditional class diagrams and feature-oriented diagrams. Figures 7 and 8 detail the analyzed questions in our questionnaire.

C. ETHICAL CONSIDERATIONS

This study did not require formal ethical approval under the University of Vale do Rio dos Sinos guidelines for research of this nature. All participants were informed of the study's objectives and procedures, and verbal informed consent was obtained prior to participation. The study was conducted in accordance with the ethical principles outlined in the Declaration of Helsinki.

D. RESULTS

We get together the collected results into two groups: *favorable* (Figure 7) and *unfavorable* (Figure 8). Those in favor indicate approval (or agreement) greater than 50%, while non-favorable indicate approval less than 50%. We focus on specific issues, including continuous reverse engineering, diagram automation, the utility of feature-oriented diagrams, the practical application of UML models, and the interplay between traditional class diagrams and feature-oriented diagrams. We evaluated each across a spectrum from *completely disagree* to *agree*, elucidating the distribution of opinions within each category.

1) CONTINUOUS REVERSE ENGINEERING

In the context of our research, 'continuous reverse engineering' refers to the ongoing process of analyzing and understanding existing software systems, often with the aid of automated tools. Our analysis revealed a notable trend towards agreement on the benefits of this approach, particularly in addressing the lack of technical documentation, with 86% and 82% of respondents expressing (partially/completely) agreement. Previous empirical studies [36], [37] have shown that widespread adoption of UML models in industry is complex and sometimes hinders. This result sheds light on the direction of overcoming this barrier. Intriguingly, the data indicates that adopting continuous reverse engineering can increase the use of UML models. Moreover, we highlight a statement from a participant obtained in the interview: *Continuous reverse engineering brings great benefits for innovation, but from the point of view of UML models, it depends on the quality of the existing code.* These findings suggest a positive inclination towards automated reverse engineering, highlighting its perceived efficacy in addressing documentation challenges and promoting the uptake of standardized modeling practices.

2) DIAGRAM AUTOMATION

Analysis of the data reveals a prevailing inclination towards the endorsement of automation, particularly in agile methodologies, where most respondents, accounting for 55%, express complete agreement on its utility. Similarly, feature-oriented diagrams are recognized for their effectiveness in increasing model adoption, with a combined 86% indicating partial or complete agreement. While there are variations in opinion, particularly within conventional methodologies, where a significant minority expresses partial disagreement, the overall trend indicates a positive reception of automated approaches and feature-oriented diagrams in software engineering practices. These findings emphasize the perceived relevance and potential benefits of incorporating automated processes and specialized diagrams in software development tasks. On the other hand, 41% disagreed that the process of automatically updating diagrams, which involves reflecting changes in the codebase in the corresponding diagrams, is helpful in the maintenance phase (Figure 8).

3) USEFULNESS OF FEATURE-ORIENTED DIAGRAM

Our data reveals a consistent trend toward agreement on the utility of feature-oriented diagrams across different contexts. The high level of agreement regarding the intention to use

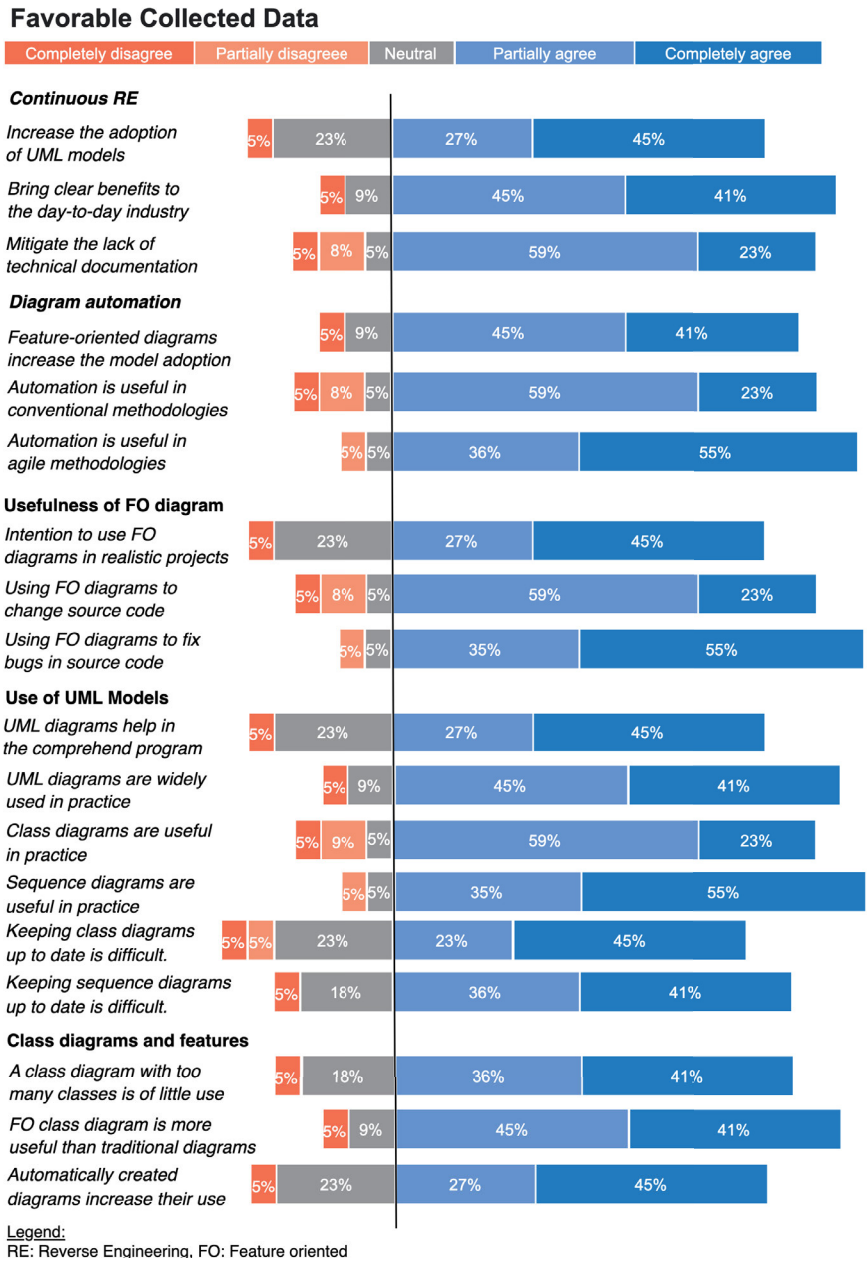


FIGURE 7. Stacked bar chart showing percentages of agreement levels for multiple survey items, ranging from completely disagree to agree, with most responses clustering in partially agree and completely agree categories.

feature-oriented diagrams in realistic projects is particularly noteworthy, with 45% of respondents expressing complete agreement. Additionally, the majority recognizes the value of these diagrams in facilitating source code changes and bug fixes, with 59% and 55%, respectively, indicating partial or complete agreement. Although there are minor variations in opinion, notably a small proportion expressing partial disagreement, the overall consensus highlights the perceived usefulness of feature-oriented diagrams for enhancing certain software development tasks. An unfavorable evaluation indicated that 86% do not desire to use feature-oriented diagrams in maintenance tasks (Figure 8). One participant highlighted that *having several class diagrams available, separated by features, brings more value than a single diagram showing the whole system. However, diagrams separated by parts that naturally go together add much value. Sequence diagrams add value by defining the actions necessary for a given implementation task. Automatically generated diagrams should be reviewed by an analyst to ensure adherence to the feature and to include secondary classes that may impact the understanding and implementation of the feature.* These findings can motivate the scientific community to

TABLE 3. Dataset presenting a 21-item questionnaire used to evaluate UMLRev qualitatively.

Focus	Question	Description
Continuous reverse engineering	Q1	Feature-driven reverse engineering integrated with continuous integration increases adoption of UML models
	Q2	Continuous reverse engineering brings clear benefits to everyday industry
	Q3	Continuous reverse engineering mitigates lack of technical documentation
Diagram automation	Q4	Resource-oriented diagrams increase model adoption
	Q5	Automation is useful in conventional methodologies
	Q6	Automation is useful in agile methodologies
	Q7	Having automatically updated diagrams is useful in the maintenance phase
Usefulness of feature oriented diagram	Q8	I intend to use resource-oriented diagrams in realistic projects
	Q9	Desire to use resource-oriented diagrams in maintenance tasks
	Q10	I intend to use feature-oriented diagrams to change source code
	Q11	I plan to use resource-oriented diagrams to fix bugs in source code
Use of UML models	Q12	UML diagrams help in understanding software
	Q13	UML diagrams are widely used in practice
	Q14	Class diagrams are useful in practice
	Q15	Sequence diagrams are useful in practice
	Q16	It is expensive to keep technical documentation up to date
	Q17	Keeping class diagrams up to date is difficult
	Q18	Keeping sequence diagrams up to date is difficult
Class diagrams and features	Q19	A class diagram with many classes is of little use
	Q20	Resource-oriented class diagram is more useful than traditional diagrams
	Q21	Automatically created diagrams increase your usage

explore the potential of feature-oriented diagrams in practical development task scenarios.

4) USE OF UML MODELS

Our findings reveal a generally positive reception towards the effectiveness and prevalence of UML diagrams in software development. Notably, a significant portion of respondents agree on the beneficial role of UML diagrams in program comprehension, with 45% indicating complete agreement. Moreover, the widespread use of UML diagrams in practice has garnered considerable endorsement, with 86% agreeing, either partially or completely, to their utility. However, the dataset highlights nuanced challenges, particularly the difficulty of keeping class and sequence diagrams up to date, with respondents expressing varying degrees of agreement regarding the associated complexities. However, 37% disagreed that maintaining technical documentation up-to-date is costly (Figure 8). These findings highlight the perceived significance of UML models in supporting developers with software development tasks, while acknowledging the inherent challenges in their maintenance, underscoring the multifaceted considerations in their practical application.

5) CLASS DIAGRAMS AND FEATURES

We also evaluated the interplay between class and feature-oriented diagrams within software engineering contexts. Our findings indicate notable trends in respondents' perceptions. Firstly, there is recognition that class diagrams with too many classes may hinder their usefulness, as evidenced by 41% who express complete agreement with this sentiment. Additionally, a majority acknowledges the enhanced utility of feature-oriented class diagrams compared to traditional ones, with 86% indicating partial or complete agreement. Furthermore, there is a widespread belief that automatically generated diagrams increase usage, with 72% of respondents either partially or completely agreeing. These findings highlight the importance of class diagram features, underscoring the need for clarity, adaptability, and efficiency in diagram creation processes to support effective software development practices.

E. LIMITATIONS

While UMLRev represents a promising step toward automating UML model extraction in continuous integration pipelines, we highlight some limitations to contextualize its contributions and outline directions for future work. These limitations pertain primarily to feature identification, the assumptions in the evaluation design, and the constraints imposed by tool-centric design choices. Clarifying these boundaries is essential for situating UMLRev's practical relevance and scientific contribution.

1) MANUAL FEATURE IDENTIFICATION

A central limitation lies in how UMLRev defines and captures "features." Currently, developers manually delimit features

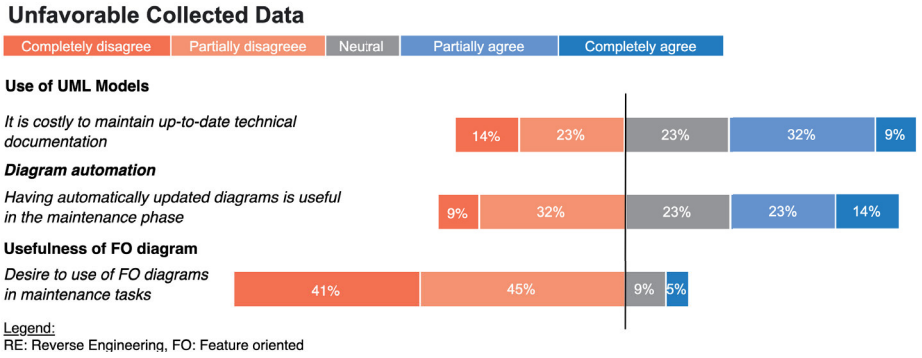


FIGURE 8. Stacked bar chart showing unfavorable survey responses, with higher proportions of completely disagree and partially disagree compared to favorable responses, alongside smaller shares of neutral, partially agree, and completely agree.

by designing test scripts. Each test script is treated as a proxy for a system feature, and the corresponding execution trace is used to extract contextual behavior. While this approach aligns with agile practices and enables human-in-the-loop modeling, it does not scale well in scenarios where test cases encompass multiple interleaved features or represent broader use-case interactions. Thus, the generated diagrams may not accurately capture fine-grained feature boundaries, resulting in either coarse or ambiguous model representations.

2) TEST SUITE DEPENDENCY AND EXECUTION TRACE FIDELITY

The effectiveness of UMLRev depends heavily on the completeness and representativeness of the integration test suite. Because execution traces derived from test scripts serve as input for reverse engineering, any omission or imbalance in the test suite may result in partial or misleading UML diagrams. Additionally, if a feature is insufficiently exercised or poorly documented in test scripts, its structural or behavioral footprint may not be captured, thereby limiting the fidelity of the resulting models. This introduces a dependency that may restrict adoption in projects with low test coverage or heterogeneous testing strategies.

3) LIMITATIONS OF THE QUALITATIVE EVALUATION

The empirical evaluation relied on qualitative feedback from 22 participants, drawn from both academia and industry. While this user-centered approach yielded rich insights into the perceived usefulness and applicability of UMLRev, the relatively small and non-random sample limits the generalizability of the findings. In particular, the evaluation does not include longitudinal or task-based measurements of effectiveness or productivity. Future work should incorporate larger, more diverse samples, mixed-methods evaluations, and comparative baselines to ensure the robustness and replicability of the results.

4) GRANULARITY AND AUTOMATION TRADE-OFFS

The current approach prioritizes automation and CI compatibility over semantic precision. While UMLRev automates model extraction based on execution traces, it does not incorporate advanced techniques such as dynamic slicing, static analysis, or trace clustering to disambiguate overlapping feature interactions. As such, the approach is best suited to modular systems with well-defined test cases. For systems with high feature coupling or unclear execution boundaries, the current model generation may oversimplify runtime behavior. This trade-off between automation and granularity presents a challenge for future work aiming to integrate deeper analysis while preserving the benefits of CI integration.

VI. DISCUSSION

A. RESEARCH AND PRACTICAL IMPLICATIONS

The research implications of the proposed approach, UMLRev, hold significant promise for advancing reverse engineering based on UML models. Addressing the persistent challenge of maintaining up-to-date UML models [36] in the context of continuous integration, UMLRev introduces a novel solution to the costly practices of creating consistent UML class diagrams and sequences, enhancing software quality. Integrating a context-aware, feature-oriented approach into the continuous integration pipeline provides a practical way to automate UML diagram generation. It opens avenues for further exploration and refinement of reverse engineering methodologies.

Researchers in the field have the opportunity to build on the foundational principles of UMLRev by exploring enhancements and extensions that cater to diverse application domains and development environments. Additionally, the qualitative empirical study underscores the importance of empirical validation in assessing the effectiveness and utility of software engineering approaches, providing researchers with valuable insights into users' perceptions and attitudes toward novel methodologies.

On a practical level, the implications of UMLRev extend beyond research, directly affecting software developers and practitioners in real-world application maintenance environments. The study's findings highlight the potential of UMLRev to catalyze the adoption of UML models within

software development teams, offering tangible benefits, including improved documentation, enhanced communication, and streamlined development workflows.

Software developers are presented with a pragmatic toolset that alleviates the burden of manual UML diagram generation, fostering a deeper understanding of system architecture and functionality. By embracing UMLRev, developers can leverage automated reverse engineering to iteratively refine and evolve UML models throughout the development process, ultimately enhancing the maintainability and scalability of software systems. As such, the practical implications of UMLRev underscore its relevance and significance for addressing critical challenges in contemporary software engineering practices.

B. LESSONS LEARNED

The proposed approach, UMLRev, yielded two valuable lessons: the importance of incorporating context awareness into reverse-engineering methodologies and the need to prioritize minimal disruption to developers' workflows for effective software modeling.

1) CONTEXT AWARENESS IN REVERSE ENGINEERING METHODOLOGIES

The proposed approach, UMLRev, offers valuable insights that can support future tool design efforts to improve software modeling processes. One key lesson from UMLRev is the importance of context awareness in reverse engineering methodologies. By leveraging contextual information derived from integration tests, UMLRev demonstrates the potential to incorporate it into the generation of UML diagrams. This emphasis on context-awareness enables UMLRev to capture the contextual relationships and dependencies inherent in evolving software systems, resulting in more accurate and valuable UML representations. Tool designers can glean from UMLRev the importance of integrating contextual awareness capabilities into software modeling tools. This enables developers to create models that more accurately reflect the dynamic nature of contemporary applications, particularly from both structural and behavioral perspectives.

2) MINIMAL DISRUPTIONS TO DEVELOPERS' WORKFLOWS

UMLRev also offers a practical lesson in the value of seamless integration with existing development workflows, such as continuous integration pipelines. By seamlessly integrating UML diagram generation into the continuous integration pipeline, UMLRev minimizes disruptions to developers' workflows and maximizes the efficiency of UML model maintenance. This perception of minimal disruptions is the main reason for the highly positive feedback from participants. This integration streamlines the reverse engineering process, ensuring that UML models remain up to date with each new source code iteration. This lesson emphasizes the importance of prioritizing integration and interoperability when designing software modeling tools, thereby aligning tool functionality with existing development practices and workflows. We emphasize that integrating modeling tools seamlessly into developers' workflows can significantly boost productivity and facilitate the adoption of best modeling practices within software development teams.

VII. CONCLUSION AND FUTURE WORK

This study presented UMLRev, a tool-supported, context-aware solution that integrates automatic UML model extraction into continuous integration (CI) pipelines. By automating the generation of class and sequence diagrams from test-driven executions, UMLRev addresses a long-standing gap in keeping architectural documentation synchronized with evolving source code. The proposed tool eliminates manual reverse engineering efforts, enabling seamless documentation updates aligned with each new software release. UMLRev is publicly available and has been designed for practical adoption in modern DevOps environments. Its architecture prioritizes real-world applicability, offering an extensible, plug-and-play design that integrates with existing CI workflows. By positioning UMLRev as a tool contribution, this work bridges a critical divide between theoretical reverse-engineering approaches and their integration into industry-ready toolchains.

To assess usability and developer perception, we conducted an empirical study with 22 practitioners. While the evaluation focused on perceived ease of use and usefulness, results demonstrate UMLRev's potential to encourage broader UML adoption and improve understanding of system behavior. Nonetheless, future studies will complement these findings with deeper comparative analyses of model quality across alternative reverse-engineering solutions.

Although UMLRev demonstrates the feasibility of continuous, feature-oriented UML reverse engineering within CI pipelines, feature identification remains an open and challenging problem. In this study, feature delimitation is performed manually by the developer, placing a human in the loop to define what constitutes a feature. This is done by creating test case scripts that allow developers to represent what their team understands of a system feature. While this pragmatic and scalable approach aligns with standard CI practices, it may not fully capture scenarios where multiple features interact within a single execution trace. Addressing fine-grained, multi-feature identification (potentially through dynamic slicing, semantic analysis of execution traces, or developer-assisted feature tagging) is a critical research challenge. We argue that advancing feature identification mechanisms is an essential next step for the scientific community to enable more precise, scalable, and semantically rich continuous reverse engineering solutions.

Finally, UMLRev provides a working prototype demonstrating the feasibility and value of integrating reverse engineering directly into CI pipelines. It marks an initial yet practical step toward industry-grade automation of model synchronization, and we hope it stimulates further innovation in tool-supported documentation and reverse engineering.

REFERENCES

[1] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. London, U.K.: Pearson, 2010.
[2] L. Chen, "Continuous delivery: Overcoming adoption challenges," *J. Syst. Softw.*, vol. 128, pp. 72–86, Jun. 2017.
[3] W. J. Dzidek, E. Arisholm, and L. C. Briand, "A realistic empirical evaluation of the costs and benefits of UML in software maintenance," *IEEE Trans. Softw. Eng.*, vol. 34, no. 3, pp. 407–432, May 2008.
[4] E. Arisholm, L. C. Briand, S. E. Hove, and Y. Labiche, "The impact of UML documentation on software maintenance: An experimental evaluation," *IEEE Trans. Softw. Eng.*, vol. 32, no. 6, pp. 365–381, Jun. 2006.
[5] H. A. Siala, K. Lano, and H. Alfraihi, "Model-driven approaches for reverse engineering—A systematic literature review," *IEEE Access*, vol. 12, pp. 62558–62580, 2024.
[6] H. A. Siala and K. Lano, "Using large language models to extract UML class diagrams from Java programs," in *Proc. 8th Int. Conf. Softw. Syst. Eng. (ICoSSE)*, Apr. 2025, pp. 70–74.
[7] W. Oizumi, A. Garcia, L. da Silva Sousa, B. Cafeo, and Y. Zhao, "Code anomalies flock together: Exploring code anomaly agglomerations for locating design problems," in *Proc. 38th Int. Conf. Softw. Eng.*, May 2016, pp. 440–451.
[8] L. Sousa, A. Oliveira, W. Oizumi, S. Barbosa, A. Garcia, J. Lee, M. Kalinowski, R. de Mello, B. Fonseca, R. Oliveira, C. Lucena, and R. Paes, "Identifying design problems in the source code: A grounded theory," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng. (ICSE)*, May 2018, pp. 921–931.
[9] T. Theunissen, U. van Heesch, and P. Avgeriou, "A mapping study on documentation in continuous software development," *Inf. Softw. Technol.*, vol. 142, Feb. 2022, Art. no. 106733.
[10] G. Canfora and M. Di Penta, "New frontiers of reverse engineering," in *Proc. Future Softw. Eng. (FOSE)*, May 2007, pp. 326–341.
[11] S. Apel and C. Kästner, "An overview of feature-oriented software development," *J. Object Technol.*, vol. 8, no. 5, p. 49, 2009.
[12] L. Briand, D. Bianculli, S. Nejati, F. Pastore, and M. Sabetzadeh, "The case for context-driven software engineering research: Generalizability is overrated," *IEEE Softw.*, vol. 34, no. 5, pp. 72–75, 2017.
[13] L. F. D'Avila, K. Farias, and J. L. V. Barbosa, "Effects of contextual information on maintenance effort: A controlled experiment," *J. Syst. Softw.*, vol. 159, Jan. 2020, Art. no. 110443.
[14] L. F. D'Avila, J. L. V. Barbosa, and K. S. F. de Oliveira, "SW-context: A model to improve developers' situational awareness," *IET Softw.*, vol. 14, no. 5, pp. 535–543, Oct. 2020.
[15] Y. Wang, L. Du, E. Shi, Y. Hu, S. Han, and D. Zhang, "CoCoGum: Contextual code summarization with multi–relational GNN on UMLs," Microsoft, Tech. Rep. MSR-TR-2020-16, 2020.
[16] K. Lano and H. Siala, "Using model-driven engineering to automate software language translation," *Automated Softw. Eng.*, vol. 31, no. 1, p. 20, May 2024.
[17] N. M. Edan and B. Mahmood, "Usage one of reverse engineering application (node.Js web) to construct software engineering documents," *Technium*, vol. 4, no. 4, pp. 1–10, May 2022.
[18] C. Pereira, L. Martinez, and L. Favre, "Toward an execution trace standard: An MDE approach," in *Proc. Argentine Congr. Comput. Sci.*, Oct. 2023, pp. 91–106.
[19] D. Rosca and L. Domingues, "A systematic comparison of roundtrip software engineering approaches applied to UML class diagram," *Proc. Comput. Sci.*, vol. 181, pp. 861–868, 2021.
[20] U. Sabir, F. Azam, S. U. Haq, M. W. Anwar, W. H. Butt, and A. Amjad, "A model driven reverse engineering framework for generating high level UML models from Java source code," *IEEE Access*, vol. 7, pp. 158931–158950, 2019.
[21] C. Baidada, E. M. Bouziane, and A. Jakimi, "A new approach for recovering high-level sequence diagrams from object-oriented applications using Petri nets," *Proc. Comput. Sci.*, vol. 148, pp. 323–332, 2019.
[22] E. Braude, "Incremental UML for agile development: Embedding UML class models in source code," in *Proc. IEEE/ACM 3rd Int. Workshop Rapid Continuous Softw. Eng. (RCoSE)*, May 2017, pp. 27–31.
[23] Y. Labiche, B. Kolbah, and H. Mehrfard, "Combining static and dynamic analyses to reverse-engineer scenario diagrams," in *Proc. IEEE Int. Conf. Softw. Maintenance*, Sep. 2013, pp. 130–139.
[24] Vinita, A. Jain, and D. K. Tayal, "On reverse engineering an object-oriented code into UML class diagrams incorporating extensible mechanisms," *ACM SIGSOFT Softw. Eng. Notes*, vol. 33, no. 5, pp. 1–9, Aug. 2008.
[25] L. C. Briand, Y. Labiche, and J. Leduc, "Toward the reverse engineering of UML sequence diagrams for distributed Java software," *IEEE Trans. Softw. Eng.*, vol. 32, no. 9, pp. 642–663, Sep. 2006.
[26] M. Merdes and D. Dorsch, "Experiences with the development of a reverse engineering tool for UML sequence diagrams: A case study in modern Java development," in *Proc. 4th Int. Symp. Princ. Pract. Program. Java*, Aug. 2006, pp. 125–134.
[27] G. Antoniol, M. Di Penta, and M. Zazzara, "Understanding web applications through dynamic analysis," in *Proc. 12th IEEE Int. Workshop Program Comprehension*, Jun. 2004, pp. 120–129.
[28] E. Stroulia and T. Systä, "Dynamic analysis for reverse engineering and program understanding," *ACM SIGAPP Appl. Comput. Rev.*, vol. 10, no. 1, pp. 8–17, Apr. 2002.
[29] H. Cabane and K. Farias, "On the impact of event-driven architecture on performance: An exploratory study," *Future Gener. Comput. Syst.*, vol. 153, pp. 52–69, 2023.
[30] M. Rubert and K. Farias, "On the effects of continuous delivery on code quality: A case study in industry," *Comput. Standards Interface*, vol. 81, Apr. 2022, Art. no. 103588.
[31] *Unified Modeling Language Specification*, Object Management Group, 2017.
[32] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*, vol. 236. Berlin, Germany: Springer, 2012.
[33] D. I. K. Sjoeberg, J. E. Hannay, O. Hansen, V. B. Kampenes, A. Karahasanovic, N.-K. Liborg, and A. C. Rekdal, "A survey of controlled experiments in software engineering," *IEEE Trans. Softw. Eng.*, vol. 31, no. 9, pp. 733–753, Sep. 2005.
[34] J. Linaaker, S. M. Sulaman, R. M. de Mello, and M. Höst, "Guidelines for conducting surveys in software engineering," Tech. Rep., 2015.
[35] J. Börstler, N. B. Ali, K. Petersen, and E. Engström, "Acceptance behavior theories and models in software engineering—A mapping study," *Inf. Softw. Technol.*, vol. 172, Aug. 2024, Art. no. 107469.
[36] E. W. Júnior, K. Farias, and B. da Silva, "On the use of UML in the Brazilian industry: A survey," *J. Softw. Eng. Res. Develop.*, vol. 10, no. 1, pp. 1–10, 2022.
[37] M. Ozkaya and F. Erata, "A survey on the practical use of UML for different software architecture viewpoints," *Inf. Softw. Technol.*, vol. 121, May 2020, Art. no. 106275.



KLEINNER FARIAS received the dual B.Sc. degree in computer science and information technology from UFAL, the M.Sc. degree in computer science from PUC-RS, and the Ph.D. degree in informatics (software engineering) from PUC-Rio, in 2012. He is currently an Associate Professor with the Applied Computing Graduate Program, University of Vale do Rio dos Sinos (UNISINOS), where he leads the Applied Software Engineering Research Group. His recent work investigates how developers manage the complexities of merging software design models, producing knowledge that supports the development of new tools, replicable studies, and actionable insights. Over his career, he has mentored more than 50 students, fostering their academic and professional development. His leadership at UNISINOS, emphasizes advancing software engineering through rigorous research, collaboration, and innovation. He has published in leading journals and conferences, including ICSE, MODELS, JSS, and *ACM Computing Surveys*. His research explores software modeling, domain-specific modeling languages, empirical software engineering, and model-driven development, with a particular focus on cognitive and collaborative aspects of software design processes. He has reviewed for top-tier venues, including ACM TOSEM, *Journal of Systems and Software*, *Empirical Software Engineering*, and SoSym.



LUCAS SCHENATTO is currently a Software Engineer skilled in both front and backend with more than nine years of experience. He has experience in building scalable and maintainable codebases and implementing responsive designs. He has worked across different industries, closely with users and product and design teams, to ensure that features meet user needs and design specifications.



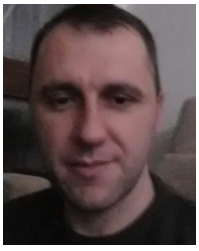
CRISTIANO FARIAS received the bachelor's degree in computer science from UNISINOS, with a focus on data processing, desktop applications, web services, and web crawling. He began as a Computer Technician and quickly developed a strong affinity for Java programming. Currently, he leads Java development for a payroll system using Java 10+, Jakarta EE, Wildfly, Spring, and VueJS. He is passionate about continuous improvement in software development and team leadership.



WILLIAN BOLZAN received the degree in computer science from the Universidade da Região da Campanha, in 2000, and the master's degree in computer science from the Pontifícia Universidade Católica do Rio Grande do Sul, in 2002. He is currently pursuing the Ph.D. degree in applied software engineering with the Graduate Program in Applied Computing, Universidade do Vale do Rio dos Sinos (UNISINOS). As a tenured Professor at the Federal Institute of Santa Catarina (IFSC), Tubarão Campus, he combines academic research with teaching, focusing on software engineering and applied computing. His work bridges theory and practice, contributing to innovation in computer science education and software development.



ROBSON KEEMPS received the degree in systems analysis from UNIVAG, in 2004, and the M.S. degree from the University of Vale do Rio dos Sinos (UNISINOS), in 2022, where he is currently pursuing the Ph.D. degree in applied computing. Previously, he was a Systems Analyst at the Mato Grosso State Department of Education (SEDUC-MT). He is a tenured Professor of computer science at the Federal Institute of Mato Grosso (IFMT), São Vicente da Serra Campus, where he integrates teaching with applied research in computing. His expertise includes GENEXUS programming, relational databases (MySQL, PostgreSQL, Oracle, and SQL Server), and business intelligence tools.



CARLOS CARBONERA received the M.Sc. degree in applied computing and the M.B.A. degree in business strategies. He is currently pursuing the Ph.D. degree in applied computing with UNISINOS, focusing on AI-assisted software engineering using retrieval-augmented generation (RAG) and large language models (LLMs). As a Full-Stack Developer and a Systems Analyst, he has more than 24 years of experience in complex corporate environments, leading multidisciplinary teams, and delivering high-impact software solutions. He specialized in C#, .NET Core, ReactJS, microservices, and database systems, with expertise in business process modeling (BPMN), ER modeling, and software architecture. He is passionate about software engineering, AI, data analysis, and bridging academic research with real-world applications.

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614